
BudgetRAG / MemAlign-Qwen

RLAIF-guided retrieval, offline bandit, and local Qwen/KV roadmap

Học chính sách chọn retrieval và cấp phát ngữ cảnh cho grounded LLM inference dưới ngân sách token, latency và KV-cache ước lượng.

Phase 1D research deck
cập nhật từ kết quả đã chạy

Câu hỏi nghiên cứu

Khi cùng một query có nhiều cách truy xuất và nhiều mức nén context,
chọn hành động nào để câu trả lời vẫn grounded với chi phí thấp hơn?

Quyết định

Retriever, fusion và context policy cùng thuộc action space.

Ngân sách

Token, latency và KV-cache là cost trực tiếp, không chỉ là log phụ.

Ranh giới claim

Policy học offline từ logged outcomes; PPO/GRPO, human preference learning và production KV pruning nằm ngoài phase này.

Chiến lược Phase 1D



State

Query features, retrieval score gap/entropy, document length statistics, retriever/model metadata.

Action

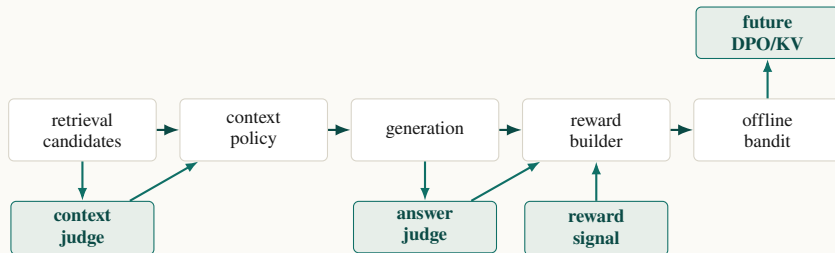
retrieval strategy + fusion + context policy + budget/adaptive profile.

Objective

Tối đa grounded quality sau khi trừ token, latency, estimated KV và lỗi generation.

Các slide kế tiếp định nghĩa action, reward và thuật toán trước; phần data chỉ dùng để kiểm tra trade-off của strategy này.

RLAIF nằm ở nhiều điểm của pipeline



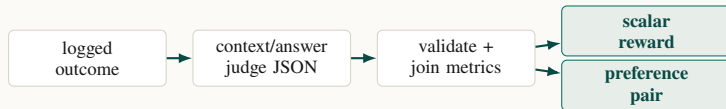
Định nghĩa dùng trong deck

RLAIF là AI feedback dùng làm nhãn chất lượng/preference. Judge chỉ nhìn question, retrieved context và answer; không browse và không dùng external knowledge.

Tác dụng với bandit

Retrieval-only metrics không biết answer có được support hay không. RLAIF thêm groundedness, unsupported-claim và evidence sufficiency vào reward.

RLAIF biến thành reward/preference như thế nào?



1. Judge label

Context judge trả về sufficiency, selected evidence, missing evidence, minimality, support và context quality. Answer judge trả về correctness, evidence support, unsupported penalty, refusal/citation/conciseness và overall quality.

2. Reward scalar

Reward builder validate score $[0, 1]$, join với token/latency/KV/error metrics, normalize cost theo run, rồi tính R . Ghi `reward_mode`: `proxy`, `rlaif_answer`, hoặc `rlaif_context_answer`.

3. Preference pair

Chỉ số sánh trong cùng query. Chosen là answer/context có RLAIF score hoặc reward cao hơn margin; rejected là missing evidence, unsupported claim, hoặc cùng support nhưng cost tệ hơn. Ties bị loại.

Bandit dùng scalar reward; DPO/ranking smoke dùng preference pairs. Cả hai đều bắt nguồn từ AI labels có cấu trúc, không phải human preference claim.

Context policy và action space

Term	Định nghĩa trong project
legacy/full	Giữ rank order và nối block context; full nghĩa là không đặt action budget riêng, chỉ chịu ceiling prompt/top-k đang có.
char-budget	Lắp một global character budget theo rank order; trim block cuối nếu vượt budget.
per-doc-budget	Cắt từng document trước rồi mới lắp global budget để tránh một document dài chiếm toàn bộ context.
score-density	Ưu tiên <code>retrieval_score</code> / <code>estimated_tokens</code> ; fallback 1 / <code>estimated_tokens</code> nếu thiếu score.
sentence-trim	Giữ rank order nhưng trim tại ranh giới câu đơn giản khi có thể.
evidence-aware	Tên CLI tương thích cũ; implementation hiện tại là lexical-query-aware : tách span, chấm bằng query overlap + retrieval score + title overlap.
adaptive-heuristic	Selector theo luật chọn fixed policy + budget dựa trên score gap, entropy và document length; đây là baseline trước bandit, chưa phải RL.

Profiles: **conservative** ưu tiên budget lớn khi retrieval không chắc; **balanced** tách ngân sách theo score gap/entropy; **aggressive** stress-test budget nhỏ hơn.

Reward: tính cụ thể từng thành phần

$$R = w_q Q_{\text{answer}} + w_s S_{\text{evident}} - w_t C_{\text{tok}} - w_l C_{\text{lat}} - w_k C_{\text{kv}} - w_e P_{\text{error}} - w_u P_{\text{unsupported}}$$

Quality/support từ RLAIIF

- ▶ Q_{answer} : overall_quality của answer judge; nếu thiếu thì dùng proxy reward và ghi rõ.
- ▶ S_{evident} : trung bình các tín hiệu evidence_support và context_quality_score.
- ▶ $P_{\text{unsupported}}$: unsupported_claim_penalty trong $[0, 1]$.
- ▶ P_{error} : 1 nếu generation/API/parsing lỗi, ngược lại 0.

Cost chuẩn hóa trong run

- ▶ C_{tok} : normalize prompt_tokens hoặc kept_est_tokens.
- ▶ C_{lat} : normalize answer_latency_s.
- ▶ C_{kv} : normalize estimated_kv_cache_mb_after.
- ▶ $\text{norm}(x) = \frac{x - \min(x)}{\max(x) - \min(x) + \varepsilon}$, theo dataset/model/run.

Default weights: $w_q = 1.0, w_s = 0.2, w_t = 0.2, w_l = 0.2, w_k = 0.1, w_e = 1.0, w_u = 1.0$. Tất cả nên là tham số CLI.

Bandit là gì trong project này?

Bandit thường

Mỗi lượt chỉ chọn một “arm” và nhận reward. Không có trajectory nhiều bước, không backprop qua môi trường, nên không gọi là full RL.

Contextual bandit

Trước khi chọn arm, policy nhìn state s : query length, score gap, entropy, doc length, retriever/model metadata.

Offline bandit

Không explore online. Policy học/evaluate từ logged outcomes đã chạy; nếu action thiếu coverage thì phải báo limitation.

Thành phần	Mapping trong BudgetRAG Phase 1D
State s	Query/retrieval features trước generation: score gap, score entropy, top-k stats, estimated tokens.
Action a	Retrieval strategy + fusion + context policy + budget/adaptive profile.
Reward r	RLAIF grounded quality trừ token, latency, estimated KV, generation error và unsupported claims.
Policy $\pi(a s)$	Hàm chọn action cho query mới; Phase 1D so sánh baseline trước rồi mới dùng learned policy.

Offline contextual bandit: thuật toán

Với mỗi query, policy chọn một action. Reward đến từ logged outcome; không tương tác online với người dùng.

Input row

query id, state features, action id, outcome metrics, optional RLAIIF labels.

Policies

So sánh
fixed/cheapest/best-average/oracle
upper bound với epsilon-greedy
replay, UCB/LinUCB và linear
reward model.

Evaluation

split train/test theo query_id;
report reward, quality, token,
latency, KV, error rate và action
distribution.

- 1. Observe
- 2. Choose
- 3. Score
- 4. Learn

tính state s trước generation từ query và retrieval candidates.
chọn $a = (retriever, fusion, context\ policy, budget, profile)$.
build reward từ RLAIIF/cost/error.
update policy offline; nếu action chưa quan sát đủ thì report limitation.

Bandit strategies sẽ đánh giá

Strategy	Vai trò trong đánh giá offline
FixedActionPolicy	Control: luôn chọn một action như bm25_evidence_4000; dùng để biết policy học có hơn baseline cố định không.
CheapestActionPolicy	Lower-cost baseline: chọn action có token/latency/KV cost thấp nhất; thường tiết kiệm mạnh nhưng dễ mất evidence.
BestAverageActionPolicy	Non-contextual baseline: chọn action có mean reward train cao nhất; bỏ qua query-specific state.
OracleLoggedPolicy	Upper bound chỉ để phân tích: chọn action tốt nhất trong logged data cho từng query; không deploy vì đã nhìn outcome.
EpsilonGreedyReplay	Replay baseline: khai thác action tốt nhất nhưng giữ ϵ xác suất explore khi log có coverage; dùng để đo trade-off exploration/exploitation offline.
UCB/LinUCB	Chọn reward estimate + uncertainty; LinUCB dùng feature vector của state nên phù hợp khi query khó cần action khác query dễ.
LinearRewardModel	Fit model dự đoán reward cho (s, a) , rồi chọn action có predicted reward cao nhất; phải report risk nếu extrapolate sang action ít log.

Kết quả chính cần báo: mean reward, quality proxy/RLAIF score, token cost, latency, estimated KV, error rate và selected action distribution.

Triển khai RLAIIF và bandit trong repo

RLAIIF path

theo Phase 1D plan

- ▶ `rlaif_schema.py`: validate context/answer judge JSON.
- ▶ `label_contexts_with_rlaif.py`: chọn evidence tối thiểu, sufficiency, minimality.
- ▶ `label_answers_with_rlaif.py`: correctness, support, hallucination/refusal/citation.
- ▶ thiếu credential thì bỏ qua có log rõ ràng; không in secret.

Bandit path

logged data trước, policy sau

- ▶ `build_budgetrag_logged_dataset.py`: gom state/action/outcome.
- ▶ `build_budgetrag_rewards.py`: normalize cost và merge RLAIIF labels.
- ▶ `retrieval_context_actions.py`: định nghĩa action set.
- ▶ `train/evaluate_budget_bandit.py`: baselines, replay, UCB, linear reward model.

Đây là contextual bandit/RL-lite. Không gọi là full RL nếu chưa có trajectory nhiều bước hoặc online policy improvement.

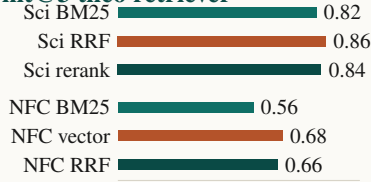
Kết quả thực nghiệm: retrieval benchmark

Dataset	Retriever	hit@3	nDCG@3	latency/q
SciFact	BM25	0.82	0.7619	0.021s
SciFact	hybrid-RRF	0.86	0.7988	0.031s
SciFact	vector-rerank	0.84	0.8011	0.028s
NFCorpus	BM25	0.56	0.3151	0.008s
NFCorpus	vector	0.68	0.3796	0.015s
NFCorpus	hybrid-RRF	0.66	0.3874	0.019s

Đọc kết quả

Hybrid/vector hữu ích trên semantic datasets; BM25 vẫn là baseline mạnh với latency thấp.

hit@3 theo retriever



LLM query rewrite/multi-query tăng latency khoảng 1.4–2.4s/query và chưa vượt BM25 trên SciFact.

Acc / quality hiện có

Retrieval accuracy proxy

Dataset	Strategy	hit@3	nDCG@3	lat/q
SciFact	BM25	0.82	0.7619	0.021s
SciFact	hybrid-RRF	0.86	0.7988	0.031s
SciFact	vector-rerank	0.84	0.8011	0.028s
NFCorpus	BM25	0.56	0.3151	0.008s
NFCorpus	vector	0.68	0.3796	0.015s
NFCorpus	hybrid-RRF	0.66	0.3874	0.019s

Đây là acc của retriever, không phải acc của final generated answer.

Một strategy “tốt” phải thắng theo acc/quality hoặc giữ quality trong ngưỡng chấp nhận được, đồng thời giảm token, latency, KV hoặc error.

Answer quality smoke, RAGAS $n = 5$

Strategy	relev.	faith.	ctx rec.
BM25	0.5106	0.5810	0.20
hybrid-RRF	0.5154	0.5133	0.00
vector-rerank	0.6992	0.6276	0.00

RAGAS ở đây là smoke sample retrieval-level; full action matrix ở các slide sau đã bổ sung R_{rel} bằng RAGAS $n = 3$ stratified.

BudgetRAG all actions: Llama8B full

Action	q	retr	R rel	err	kept tok	KV MB	lat
legacy 1k	50	.84/.762	.274	0	250.4	1714	4.9s
legacy 2k	50	.84/.762	.630	0	501.7	1479	8.4s
legacy 4k	50	.84/.762	.617	0	1003.9	1008	15.6s
legacy 8k	50	.84/.762	.605	0	1881.1	186	27.7s
evidence 1k	50	.84/.762	.316	0	250.0	1715	6.2s
evidence 2k	50	.84/.762	.628	0	500.0	1480	8.5s
evidence 4k	50	.84/.762	.613	0	1000.0	1012	14.4s
evidence 8k	50	.84/.762	.535	0	1999.9	189	28.8s
adapt-bal 1k	50	.84/.762	.522	0	565.0	1419	11.0s
adapt-bal 2k	50	.84/.762	.628	0	710.0	1283	12.1s
adapt-bal 4k	50	.84/.762	.950	25	1000.0	1012	33.8s
adapt-bal 8k	50	.84/.762	.581	0	1580.0	536	22.9s
adapt-aggr 1k	50	.84/.762	.266	0	250.0	1715	6.2s
adapt-aggr 2k	50	.84/.762	.617	0	340.0	1630	7.4s
adapt-aggr 4k	50	.84/.762	.634	34	520.0	1462	25.3s
adapt-aggr 8k	50	.84/.762	.318	2	843.0	1181	14.5s

R rel=mean RAGAS answer relevancy trên 3 cached answers/action row; sampling spread, không lọc noncommittal. Đây là answer-quality proxy, chưa phải label accuracy.

BudgetRAG all actions: Qwen3-32B full

Action	q	retr	R rel	err	kept tok	KV MB	lat
legacy 1k	50	.84/.762	.000	0	250.4	1714	6.1s
legacy 2k	50	.84/.762	.318	0	501.7	1479	8.4s
legacy 4k	50	.84/.762	.312	0	1003.9	1008	14.5s
legacy 8k	50	.84/.762	.630	0	1881.1	186	27.8s
evidence 1k	50	.84/.762	.312	1	250.0	1715	6.3s
evidence 2k	50	.84/.762	.638	1	500.0	1480	8.6s
evidence 4k	50	.84/.762	.630	0	1000.0	1012	14.5s
evidence 8k	50	.84/.762	.630	0	1999.9	189	28.8s
adapt-bal 1k	50	.84/.762	.324	1	565.0	1419	9.8s
adapt-bal 2k	50	.84/.762	.318	0	710.0	1283	11.0s
adapt-bal 4k	50	.84/.762	.630	0	1000.0	1012	14.6s
adapt-bal 8k	50	.84/.762	.303	28	1580.0	536	54.1s
adapt-aggr 1k	50	.84/.762	.312	1	250.0	1715	6.1s
adapt-aggr 2k	50	.84/.762	.312	1	340.0	1630	7.3s
adapt-aggr 4k	50	.84/.762	.318	0	520.0	1462	9.7s
adapt-aggr 8k	50	.84/.762	.282	34	843.0	1181	36.2s

R **rel**=mean RAGAS answer relevancy $n = 3$. Qwen legacy 1k vẫn thấp vì 3 valid spread samples đều bị MiMo judge coi là không liên quan/noncommittal.

BudgetRAG all actions: MiMo full

Action	q	retr	R rel	err	kept tok	KV MB	lat
legacy 1k	50	.84/.762	.310	0	250.4	1714	7.9s
legacy 2k	50	.84/.762	.564	0	501.7	1479	8.0s
legacy 4k	50	.84/.762	.920	0	1003.9	1008	8.2s
legacy 8k	50	.84/.762	.814	0	1881.1	186	8.5s
evidence 1k	50	.84/.762	.294	0	250.0	1715	8.1s
evidence 2k	50	.84/.762	.315	0	500.0	1480	8.3s
evidence 4k	50	.84/.762	.553	0	1000.0	1012	8.5s
evidence 8k	50	.84/.762	.104	0	1999.9	189	8.7s
adapt-bal 1k	50	.84/.762	.576	0	565.0	1419	8.7s
adapt-bal 2k	50	.84/.762	.939	0	710.0	1283	9.2s
adapt-bal 4k	50	.84/.762	.565	0	1000.0	1012	9.5s
adapt-bal 8k	50	.84/.762	.897	0	1580.0	536	9.7s
adapt-aggr 1k	50	.84/.762	.327	0	250.0	1715	8.9s
adapt-aggr 2k	50	.84/.762	.931	0	340.0	1630	9.2s
adapt-aggr 4k	50	.84/.762	.570	0	520.0	1462	9.6s
adapt-aggr 8k	50	.84/.762	.907	0	843.0	1181	9.7s

R rel=mean RAGAS answer relevancy trên 3 cached answers/action row; sampling spread, không lọc noncommittal. Đây là answer-quality proxy, chưa phải label accuracy.

BudgetRAG all actions: MiMo long context

Action	q	retr	R rel	err	kept tok	KV MB	lat
legacy 4k	30	.90/.742	.000	0	1003.9	2845	11.3s
legacy 8k	30	.90/.742	.272	0	2008.4	1903	11.1s
legacy 16k	30	.90/.742	.499	0	3766.4	255	12.3s
legacy 32k	30	.90/.742	.543	0	4038.2	0	12.5s
evidence 4k	30	.90/.742	.273	0	1000.0	2848	11.5s
evidence 8k	30	.90/.742	.580	0	2000.0	1911	11.3s
evidence 16k	30	.90/.742	.610	0	3999.9	260	12.9s
evidence 32k	30	.90/.742	.910	0	6529.2	0	12.3s
adapt-bal 4k	30	.90/.742	.318	0	1000.0	2848	11.2s
adapt-bal 8k	30	.90/.742	.291	0	1466.6	2411	11.3s
adapt-bal 16k	30	.90/.742	.314	0	2399.9	1668	10.8s
adapt-bal 32k	30	.90/.742	.317	0	3448.7	1583	10.9s
adapt-aggr 4k	30	.90/.742	.266	0	550.0	3270	10.0s
adapt-aggr 8k	30	.90/.742	.309	0	950.0	2895	10.5s
adapt-aggr 16k	30	.90/.742	.270	0	1630.1	2297	10.2s
adapt-aggr 32k	30	.90/.742	.221	0	2577.0	2242	10.6s

R rel=mean RAGAS answer relevancy trên 3 cached answers/action row; sampling spread, không lọc noncommittal. Đây là answer-quality proxy, chưa phải label accuracy.

RAGAS protocol: đã fix sampling

Dataset fields

- ▶ `user_input`: SciFact claim/query.
- ▶ `response`: cached generated answer của action.
- ▶ `retrieved_contexts`: giữ để trace và chạy faithfulness riêng nếu cần.

Metric + sampling

- ▶ Slide dùng `answer_relevancy`; không giả lập reference cho correctness.
- ▶ Mỗi action row có 3 valid cached answers, chọn spread deterministic.
- ▶ Không lọc câu noncommittal; timeout được thay bằng candidate tiếp theo.

Artifact mới đạt 64/64 rows với $n = 3$. Muốn answer correctness thật cần thêm gold free-text answer hoặc stance judge theo label SciFact.

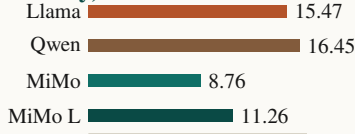
Kết quả thực nghiệm: generation cost/error matrix

Run	rows	err.	latency	prompt tok.
Groq Llama 8B full	800	61	15.47s	903.7
Groq Qwen3-32B full	800	67	16.45s	903.7
MiMo v2.5 Pro full	800	0	8.76s	903.7
MiMo long top-k 10	480	0	11.26s	2477.0

Tổng hợp context/KV

Full matrix giữ cùng trung bình 824.7 context tokens; KV sau budget khoảng 760 MB, tiết kiệm khoảng 1189 MB. Các slide trước bổ sung $rel\ n = 3$ làm answer-quality proxy.

Latency, seconds



Long-context stress test

MiMo long-context giữ trung bình 2398 context tokens; KV sau budget khoảng 1959 MB, tiết kiệm khoảng 1827 MB.

HotpotQA eval: chuyển sang Kaggle

Vì sao không chạy local matrix

- ▶ HotpotQA BEIR có khoảng 5.23M documents; smoke local phải tải 654 MB và build docstore rất lâu.
- ▶ BM25 local dùng khoảng 8–9 GB RSS trước generation.
- ▶ Matrix cũ sẽ rebuild index theo từng action row, nên không phù hợp để chạy 16–64 rows.

Pipeline mới

- ▶ Kaggle private notebook, CPU, internet enabled, MiMo secret qua `MIMO_API_KEY`.
- ▶ Build BM25 một lần rồi ghi `retrieval_cache.jsonl`.
- ▶ Replay 16 MiMo action rows: legacy, evidence-aware, adaptive balanced/aggressive ở 4k–32k chars.
- ▶ Join `hotpotqa/hotpot_qa` để có EM/token-F1; RAGAS post-hoc $n = 5/\text{action}$.

Slide HotpotQA sẽ lấy số từ `benchmark_results/budgetrag/phase1c3_hotpotqa_kaggle/<run>/hotpotqa_summary.csv`; chứa trộn với SciFact matrix vì khác benchmark và khác sampling.

Trade-off: làm sao biết strategy nào tốt?

Đã có trong deck

- ▶ Retrieval benchmark có accuracy proxy: hit@3 , nDCG@3 , recall/precision.
- ▶ RAGAS smoke $n = 5$ có answer/context proxy cho retrieval-level.
- ▶ Generation matrix đã có $R_{\text{rel } n = 3}$ stratified cho toàn bộ 64 action rows, nhưng vẫn là proxy nhỏ.

Cách chọn sau khi có labels

Pareto	action tốt nếu quality không giảm quá δ nhưng token/latency/KV thấp hơn.
Reward	chọn action có $R = \text{quality} + \text{support} - \text{cost} - \text{error}$ cao nhất trên validation queries.
Robustness	kiểm tra error rate, provider quota, và action distribution; không chọn action chỉ thắng nhờ thiếu coverage.
Generalize	train/test split theo <code>query_id</code> ; báo riêng SciFact/NFCorpus nếu có đủ run.

Vì vậy Phase 1C.3 đã có cost/error và answer-quality proxy nhỏ. Phase 1D RLAIIF mới biến proxy đó thành reward ổn định để chọn policy.

Data đọc như thế nào cho policy

Retrieval action

SciFact cho thấy BM25 vẫn là action rẻ và mạnh; hybrid/vector chỉ đáng chọn khi reward bù được build/latency cost.

Context action

Phase 1C.3 đã có RAGAS answer relevancy $n = 3$ stratified cho matrix; reward vẫn cần thêm evidence sufficiency và groundedness ổn định hơn.

Generator action

MiMo long-context là upper bound; Groq/Qwen high-context adaptive bị rate limit nên error penalty và quota provider phải nằm trong reward.

Kết luận không phải “policy nhỏ nhất luôn tốt”. Một action chỉ tốt nếu nằm trên Pareto frontier: grounded quality giữ được, cost và lỗi giảm.

Qwen2.5 KV-cache @ 1k / 16k / 128k

Model	layers	KV heads	head dim	1k	16k	128k
Qwen2.5-0.5B	24	2	64	12 MiB	192 MiB	1.5 GiB
Qwen2.5-1.5B	28	2	128	28 MiB	448 MiB	3.5 GiB
Qwen2.5-3B	36	2	128	36 MiB	576 MiB	4.5 GiB
Qwen2.5-7B	28	4	128	56 MiB	896 MiB	7.0 GiB
Qwen2.5-14B	48	8	128	192 MiB	3.0 GiB	24.0 GiB

$$KV = 2 \times layers \times num_key_value_heads \times head_dim \times seq \times batch \times dtype_bytes$$

Assumption: batch=1, fp16/bf16. 1k = 1024 tokens, 16k = 16384, 128k = 131072. KV tăng tuyến tính theo sequence length. Bảng dùng KV heads/GQA; các run cũ dùng profile analytical qwen2.5-14b conservative hơn.

Local Qwen và roadmap KV

Estimator/profiling

- ▶ đọc model config, không load weights cho estimate;
- ▶ start Qwen2.5 0.5B/1.5B nếu có GPU;
- ▶ đo prompt tokens, output tokens, prefill/decode latency, peak CUDA memory, estimated KV.

KV pruning POC

- ▶ prefill full/compressed context;
- ▶ capture `past_key_values`;
- ▶ retain sink + recent + RLAIIF evidence tokens;
- ▶ slice KV rồi decode continuation.

Phase 1D chỉ bao gồm estimate/profiling và POC nếu chạy thật; production KV-cache pruning nằm ngoài phạm vi.

Kế hoạch thực thi 7 ngày

Ngày	Trọng tâm	Kết quả cần có
1	Thu generation outputs	Phase 1C.3 logs, logged outcomes
2	Context RLAIIF	evidence chunks/spans, preference pairs
3	Answer RLAIIF	correctness/support/hallucination/refusal/citation labels
4	Reward	normalized token/latency/KV costs, reward JSONL
5	Bandit	baselines + bandit eval against fixed/adaptive heuristic
6	Local Qwen/KV	KV estimate table, small-model profiling if GPU available
7	Report	Phase 1D report, slide update, quyết định bước tiếp theo

Quyết định tiếp theo: runtime KV pruning POC hay DPO smoke, dựa trên evidence labels và profiling thật.

Ranh giới claim

Có thể nói

- ▶ đã có fixed/adaptive BudgetRAG policy và KV estimator;
- ▶ Phase 1C.3 snapshot có số liệu token/KV saving;
- ▶ Phase 1D đề xuất RLAIIF + offline contextual bandit;
- ▶ appendix KV quantization chỉ tính payload từ bit-width.

Không nói nếu chưa chạy

- ▶ đã có PPO/GRPO hoặc full RL;
- ▶ có human preference learning;
- ▶ Qwen14B DPO đã hoàn tất;
- ▶ runtime KV pruning production-ready;
- ▶ KV quantization latency/quality/VRAM overhead khi chưa đo.

Deck này cố ý tách kết quả đã chạy, tính toán analytical và roadmap triển khai để tránh claim quá mức.

Kết thúc

Luận điểm chính

- ▶ action space đủ hẹp để chạy offline bandit, nhưng vẫn bao phủ retriever, fusion, policy và budget;
- ▶ RLAIIF biến “context có vẻ tốt” thành label đo được: evidence đủ, answer được support, claim không bịạ;
- ▶ KV-cache được đưa vào reward như cost, chưa claim runtime pruning khi chưa đo.

Quyết định tiếp theo

- ▶ chốt label schema cho context/answer RLAIIF;
- ▶ chạy logged-outcome matrix đủ lớn cho bandit eval;
- ▶ profile local Qwen trước khi nói về runtime KV pruning hoặc TurboQuant gain thật.

Chốt: học policy từ logged outcomes; giữ groundedness làm điều kiện, tối ưu token/latency/KV như cost.

Appendix: KV quantization payload estimate

Công thức payload

$$TQ_{3.5b} = KV_{bf16} \times \frac{3+4}{16+16} = KV_{bf16} \times \frac{7}{32}$$

Giả định variant 3-bit key + 4-bit value. Đây chỉ là payload KV, chưa cộng metadata, packing alignment, kernel workspace hay allocator overhead.

Ví dụ kiểm tra

Qwen2.5-14B 16k bf16 KV = 3072 MiB.
 $3072 \times 7/32 = 672$ MiB.

Qwen2.5 KV payload nếu TurboQuant 3.5-bit

Model	16k TQ payload	128k TQ payload
Qwen2.5-0.5B	42 MiB	336 MiB
Qwen2.5-1.5B	98 MiB	784 MiB
Qwen2.5-3B	126 MiB	1008 MiB
Qwen2.5-7B	196 MiB	1568 MiB
Qwen2.5-14B	672 MiB	5376 MiB

128k Qwen2.5-14B: $24576 \times 7/32 = 5376$ MiB. Không dùng headline 6x hoặc runtime savings khi chưa đo implementation cụ thể.